PY Live-Coding Cheatsheet

HackerRank-style interviews · syntax, builtins, containers, sort, strings, itertools · one A3 sheet

syntax key: `keyword` `int Type` `"str"` `builtin` `# comment`
★ use constantly · TRAP common gotcha · ▌ trap / tip

I/O 01–03 · Syntax 04–06 · Collections 07–17 · Types & Errors 15–16 · Concurrency 18 · Algorithms 19–20

sections numbered 01→ in reading order, column by column

01 Fast I/O — sys.stdin beats input() — I/O

```
import sys
input = sys.stdin.readline                # 10x faster than input() ★
n = int(input())
a = list(map(int, input().split()))
x, y = map(int, input().split())          # "3 5"
s = input().rstrip()                      # readline keeps '\n' ★
grid = [list(input().rstrip()) for _ in range(n)]       # char grid
mat = [list(map(int, input().split())) for _ in range(n)]
# everything at once (fastest)
data = sys.stdin.read().split(); it = iter(data)
n = int(next(it)); a = [int(next(it)) for _ in range(n)]
# buffered output ★
out = []
out.append(f"{n} {a[0]}")
sys.stdout.write("\n".join(out) + "\n")      # or print(*out, sep="\n")
print(*a)                                  # "1 2 3"
print(" ".join(map(str, a)))               # same
print(f"{ans:.6f}")   print("YES" if ok else "NO")
print(x, end="")   print(a, b, sep=", ")   print(x, file=sys.stderr)
sys.setrecursionlimit(1 << 25)             # deep recursion ★
```

Traps `input()` strips the newline, `sys.stdin.readline()` does NOT → `rstrip()` for strings. Empty line at EOF → `""`. `print` in a loop of 10^5 lines is slow — collect then write once.

02 HackerRank Input Patterns — stubs — I/O

```
# q queries: "push 5" / "pop" / "print"
for _ in range(int(input())):
    cmd, *args = input().split()           # star-unpack ★
    if cmd == "push": st.append(int(args[0]))
    elif cmd == "pop": st.pop()
    else: print(st[-1])
# until EOF
for line in sys.stdin:
    line = line.rstrip()
    if not line: continue
# header "n m" then n rows
n, m = map(int, input().split())
rows = [input().split() for _ in range(n)]
# parse mixed line "alice 30 5.5"
name, age, score = input().split(); age = int(age); score = float(score)
# output a matrix
for row in mat: print(*row)
# function signature stub
def solve(n: int, a: list[int]) -> int:
    return sum(a)
if __name__ == "__main__": print(solve(n, a))
```

03 f-strings · Formatting — I/O

```
f"{x}"  f"{x!r}"  f"{x=}"                 # repr / debug "x=5"
f"{n:5}|{n:<5}|{n:^5}|{n:>5}"             # width / align
f"{n:05d} {f:.3f} {f:8.2f} {n:+d} {n:,}"  # pad, precision, sign, 1,000
f"{n:b} {n:o} {n:x} {n:X} {n:#x} {f:e} {p:.1%}"
f"{x:>{w}}"                               # width from variable
"{} {}".format(a, b)   "%d %s" % (n, s)   # older styles
str(x)  repr(x)  int("42")  float("3.5")  int("ff", 16)  bin(5)  hex(255)
```

04 Variables · Types · Numbers — SYNTAX

```
x = 5;  y = 3.0;  s = "hi";  ok = True;  z = None;  big = 10**18  # no overflow ★
a, b = 1, 2;  a, b = b, a                 # tuple assign / swap ★
x = y = 0;  a, *rest = [1, 2, 3];  first, *_, last = seq
MOD = 10**9 + 7;  INF = float("inf")
7 // 2 == 3   -7 // 2 == -4 (floors!)   7 / 2 == 3.5   -7 % 3 == 2 (non-neg)
int(-3.7) == -3   round(2.5) == 2 (banker's)   math.floor  math.ceil
divmod(7, 2)  abs(x)  pow(b, e, m)  x ** 0.5  math.isqrt(n)  math.gcd(a, b)
max(a, b)  min(a, key=f)  sum(a)  sum(a, start)  math.inf  math.comb(n, k)
isinstance(x, int)  type(x) is int   int("12") == 12   "12" + "3" == "123"
bool(0) bool("") bool([]) bool(None) → False;  everything else True ★
x = 5 if cond else 6   (a or b)  (a and b)   not x   x is None   x in s
```

Trap `is` compares identity, `==` value. Use `is None`. Ints are arbitrary precision — but floats are IEEE: `0.1 + 0.2 != 0.3`, use `Fraction` / `Decimal` or ints.

05 Control Flow — SYNTAX

```
if x > 0: ...
elif x < 0: ...
else: ...
for i in range(n): ...          for i in range(n-1, -1, -1): ...  # reverse
for i in range(0, n, 2): ...    for i, v in enumerate(a, start=1): ...
for v in a: ...   for k, v in d.items(): ...   for a, b in zip(x, y): ...
while cond: ...   while True: ... break
for ..: ... else: ...           # else runs only if loop did NOT break ★
break   continue   pass
# no labeled break: use a flag, a function + return, or raise
match cmd:                      # structural pattern matching (3.10+)
    case ["push", x]: ...
    case ["pop"]: ...
    case [a, b, *rest] if a < b: ...
    case {"k": v}: ...
    case _: ...
with open(p) as f: ...          # context manager
```

06 Functions · Lambdas · Scope — SYNTAX

```
def f(a, b=2, *args, key=None, **kw): return a + b      # defaults, varargs
f(1, 2, 3, key=4)   f(*lst)   f(**dct)                  # unpack
def divmod_(a, b): return a // b, a % b                 # tuple return
q, r = divmod_(7, 2)
sq = lambda x: x * x;   key=lambda p: (-p[1], p[0])
def outer():
    cnt = 0
    def inc(): nonlocal cnt; cnt += 1   # write to enclosing scope ★
g = 0
def h(): global g; g += 1               # write to module scope
from functools import lru_cache, cache, reduce, partial
@lru_cache(maxsize=None)                # memoize recursion ★ (args hashable)
def fib(n): return n if n < 2 else fib(n-1) + fib(n-2)
reduce(lambda a, b: a * b, xs, 1)       partial(f, 1)
def gen(n):                             # generator: lazy ★
    for i in range(n): yield i * i
list(gen(5))   next(g)   sum(x for x in gen(5))
def f(x: int, a: list[int]) -> int: ...  # type hints (ignored at runtime)
```

Trap Mutable default arg `def f(x=[])` is shared across calls → use `None`. Assigning to a name inside a function makes it local for the whole function (UnboundLocalError).

07 list · Basics — COLLECTIONS

```
a = [];  a = [1, 2, 3];  a = [0] * n;  a = list(range(n));  a = [None] * n
g = [[0] * m for _ in range(n)]         # 2D ★
g = [[0] * m] * n                       # BUG: all rows are the SAME list ★
a.append(x)  a.pop()  a.pop(0)/*O(n)*/  a.insert(i, x)  a.extend(b)  a += b
len(a)  a[i]  a[-1]  a[-2]  a.index(x)  a.count(x)  x in a  /*O(n)*/
a.remove(x)  del a[i]  del a[i:j]  a.clear()
a.reverse()  a.sort()  reversed(a)  sorted(a)   # in place vs new
max(a)  min(a)  sum(a)  a.index(max(a))   # argmax
a[i], a[j] = a[j], a[i]                 # swap
b = a[:]  b = a.copy()  b = list(a)     # shallow copy; `b = a` aliases ★
import copy; copy.deepcopy(g)           # nested copy
[x * 2 for x in a if x > 0]             # list comprehension ★
[f(x) for row in g for x in row]        # flatten
list(zip(*g))                           # transpose ★
any(x > 0 for x in a)  all(..)  sum(1 for x in a if p(x))
```

08 Slicing · Concat · Conversion — COLLECTIONS

```
a[i:j]  a[:k]  a[k:]  a[::2]  a[::-1]  a[-3:]      # ALWAYS a new list ★
a[i:j] = [..]   a[::2] = [0] * len(a[::2])         # slice assign
s[1:3]  s[::-1]                                    # strings too
c = a + b                                          # new list
a += b   a.extend(b)                               # in place
a = a + b   # rebinds a (new object) — differs from += when aliased ★
flat = [x for row in g for x in row]
flat = list(itertools.chain.from_iterable(g))
list("abc") → ['a','b','c']    "".join(chars) → "abc"
list(range(5))  tuple(a)  set(a)  list(s)  dict(pairs)  list(d.items())
list(map(int, strs))   list(map(str, nums))   [int(x) for x in strs]
s.split()  s.split(",")   ",".join(map(str, a))
s.encode()  b.decode()  bytes([65, 66])  bytearray(b"ab")  list(b"ab")
a == b  (lists compare elementwise)   (1, 2) < (1, 3)   "abc" < "abd"
```

09 Tuples · Sets · Unpacking — COLLECTIONS

```
t = (1, 2);  t = 1, 2;  single = (1,);  t[0];  x, y = t      # immutable, hashable
s = set();  s = {1, 2};  set(a)                               # {} is a dict!
s.add(x)  s.discard(x)  s.remove(x)/*KeyError*/  x in s  /*O(1)*/  len(s)
a | b  a & b  a - b  a ^ b  a <= b  a.issubset(b)  s.pop()
frozenset(a)                    # hashable set (set-of-sets, dict key)
seen = set(); if x in seen: ..; seen.add(x)
{x % 3 for x in a}              # set comprehension
pts = {(r, c)}                  # tuples as coords ★; lists are unhashable
a, b, *rest = [1, 2, 3, 4]      # rest = [3, 4]
for i, (x, y) in enumerate(pairs): ..
k, v = max(d.items(), key=lambda kv: kv[1])
```

10 dict · defaultdict · Counter — COLLECTIONS

```
d = {};  d = {"a": 1};  d = dict(a=1);  d = dict(zip(keys, vals))
d[k] = v;  d[k]  /*KeyError*/   d.get(k)  d.get(k, 0)   k in d   len(d)
del d[k]   d.pop(k)   d.pop(k, None)   d.setdefault(k, []).append(x)
d[k] = d.get(k, 0) + 1                  # counting (plain dict)
for k in d:  for k, v in d.items():  d.keys()  d.values()  list(d)
sorted(d)  sorted(d.items(), key=lambda kv: -kv[1])   max(d, key=d.get)
{k: v for k, v in zip(a, b)}  {v: k for k, v in d.items()}   # invert
d.update(e)   d | e  (3.9+)   d.items() preserves insertion order ★
from collections import defaultdict, Counter, OrderedDict
dd = defaultdict(int);  dd[k] += 1                  # counting ★
dd = defaultdict(list); dd[u].append(v)             # adjacency ★
dd = defaultdict(lambda: defaultdict(int))          # nested
cnt = Counter(s);  cnt["a"]  cnt.most_common(3)  cnt.update(t)  cnt1 + cnt2
Counter(a) == Counter(b)                # anagram / same multiset ★
keys must be hashable: str, int, tuple ok — list / set / dict NOT ★
```

Trap `defaultdict` creates the key on READ: `if dd[k]` inserts `k`. Use `k in dd`. Iterating a dict while changing its size → RuntimeError: iterate over `list(d)`.

11 Sorting · bisect — COLLECTIONS

```
a.sort()   b = sorted(a)   a.sort(reverse=True)            # stable ★
a.sort(key=lambda p: p[1])   a.sort(key=len)   a.sort(key=str.lower)
a.sort(key=lambda p: (-p[1], p[0]))         # score desc, name asc ★
a.sort(key=lambda p: (p[1], p[0]), reverse=True)
from operator import itemgetter, attrgetter
a.sort(key=itemgetter(1, 0))   objs.sort(key=attrgetter("age"))
from functools import cmp_to_key
a.sort(key=cmp_to_key(lambda x, y: (x > y) - (x < y)))   # custom cmp
sorted(d.items(), key=lambda kv: kv[1])    sorted(zip(a, b))
idx = sorted(range(n), key=lambda i: a[i])  # argsort
max(a, key=f)  min(a, key=f)  heapq.nlargest(k, a, key=f)
import bisect
bisect.bisect_left(a, x)   # lower_bound: first i with a[i] >= x ★
bisect.bisect_right(a, x)  # upper_bound: first i with a[i] > x
bisect.insort(a, x)        # insert keeping sorted (O(n))
bisect.bisect_left(a, x, key=f)   (3.10+)
```

12 Strings — immutable — every op makes a new str — COLLECTIONS

```
s = "hello";  len(s)  s[0]  s[-1]  s[1:3]  s[::-1]  /*reverse ★*/
s.upper() .lower() .capitalize() .title() .swapcase() .strip() .lstrip(" x")
s.split()  s.split(",")  s.split(",", 1)  s.splitlines()  s.partition("=")
" ".join(words)   "".join(chars)   ",".join(map(str, nums))     # list -> str ★
s.replace("a", "b")  s.replace("a", "b", 1)   s.count("ab")   s * 3
s.find("x") /*-1*/  s.index("x") /*ValueError*/  s.rfind  "x" in s
s.startswith("ab")  s.endswith(("x", "y"))   s.startswith(prefix, i)
s.isdigit() .isalpha() .isalnum() .isspace() .isupper() .islower()
s.zfill(5)  s.ljust(5, "*")  s.rjust(5)  s.center(7)  s.removeprefix("ab")
ord("a") == 97   chr(98) == "b"   ord(c) - ord("a")   chr(ord("a") + i)
s == t   s < t  (lexicographic)   sorted(s)   "".join(sorted(s))  # anagram key
list(s)   s.encode()   str(n)   int(s)   s.isdigit() and int(s)
import string; string.ascii_lowercase  .ascii_uppercase  .digits  .punctuation
s.translate(str.maketrans("abc", "xyz"))   s.translate(str.maketrans("", "", ",."))
import re; re.findall(r"\d+", s)  re.sub(r"\s+", " ", s)  re.match  re.split
```

13 String Building · bytes · bytearray — append patterns — COLLECTIONS

```
parts = []; parts.append("x"); parts.append(str(n)); s = "".join(parts)  # ★
s += "x"                                # O(n) each — fine for small, join for loops
s = "".join(f(x) for x in a)   s = "".join(map(str, a))
from io import StringIO; buf = StringIO(); buf.write("ab"); buf.getvalue()
print(*parts, sep="")                   # or write directly
# bytes: immutable;  bytearray: mutable ★
b = b"abc";  b[0] == 97  /*int!*/   b[0:1] == b"a"   b.decode()  "é".encode()
ba = bytearray();  ba += b"ab";  ba.append(65);  ba.extend(b"cd")
ba[0] = 66;  bytes(ba)  ba.decode()  len(ba)  ba[::-1]
ba = bytearray(n)                       # n zero bytes;  bytearray("hi", "utf8")
int.from_bytes(b, "big")   n.to_bytes(4, "little")   bytes.fromhex("ff")  b.hex()
cs = list(s); cs[i] = "x"; s = "".join(cs)      # "mutate" a string ★
arr = [0] * 26; arr[ord(c) - 97] += 1           # letter counts
import array; array.array("i", [1, 2])          # compact typed array
```

14 Comprehensions · itertools · Builtins — COLLECTIONS

```
[x for x in a]  {x for x in a}  {k: v for ..}  (x for x in a)  /*generator*/
[x if x > 0 else 0 for x in a]   [x for x in a if x > 0]
[(i, j) for i in range(n) for j in range(i + 1, n)]        # all pairs i<j
enumerate(a)  zip(a, b)  zip(*pairs)  reversed(a)  range(a, b, step)
map(f, a)  filter(p, a)  sum  min  max  any  all  sorted  len  abs  round
from itertools import *
permutations(a, r)  combinations(a, r)  combinations_with_replacement(a, r)
product(a, b)  product(range(3), repeat=2)   accumulate(a)  /*prefix sums ★*/
chain(a, b)  chain.from_iterable(g)  groupby(sorted(a))  pairwise(a)  /*3.10*/
islice(it, k)  count(0)  cycle(a)  repeat(x, n)  zip_longest(a, b, fillvalue=0)
from collections import deque, defaultdict, Counter, namedtuple
from functools import lru_cache, reduce, cmp_to_key
import math, heapq, bisect, sys, itertools, collections, string, re
```

15 Classes · dataclass · dunder — TYPES & ERRORS

```
class P:
    def __init__(self, x, y=0): self.x, self.y = x, y
    def dist(self): return abs(self.x) + abs(self.y)
    def __repr__(self): return f"P({self.x}, {self.y})"
    def __eq__(self, o): return (self.x, self.y) == (o.x, o.y)
    def __hash__(self): return hash((self.x, self.y))      # for set / dict key
    def __lt__(self, o): return (self.x, self.y) < (o.x, o.y)  # sort / heap ★
    @staticmethod
    def origin(): return P(0, 0)
    @property
    def norm(self): return self.dist()
class Dog(Animal):
    def __init__(self, n): super().__init__(n)
    def speak(self): return "woof"
from dataclasses import dataclass, field
@dataclass(order=True, frozen=True)          # eq, repr, lt, hash for free ★
class P: x: int; y: int = 0; tags: list = field(default_factory=list)
Pt = namedtuple("Pt", "x y");  p = Pt(1, 2);  p.x;  p._replace(x=3)
__len__ __getitem__ __iter__ __contains__ __str__ __call__ __enter__ __exit__
```

16 Errors · None · Truthiness — TYPES & ERRORS

```
try:
    n = int(s)
except ValueError as e:        # except (A, B) as e
    print(e)
else: ...                      # ran without exception
finally: ...                   # always
raise ValueError("bad")   raise   /*re-raise*/   raise X from e
class MyErr(Exception): pass
KeyError IndexError ValueError TypeError ZeroDivisionError RecursionError
x = d.get(k)   if x is None: ..   x = d.get(k) or default   # 0 / "" are falsy! ★
v = a[i] if i < len(a) else None
assert cond, "msg"             # stripped with -O
try: v = a[i]
except IndexError: v = None
import sys; sys.exit(0)        os._exit   quit()
```

17 heapq · deque · Queue — COLLECTIONS

```
import heapq                   # min-heap on a plain list ★
h = []; heapq.heappush(h, x); x = heapq.heappop(h); h[0] /*peek*/; len(h)
heapq.heappush(h, -x); -heapq.heappop(h)              # max-heap trick ★
heapq.heappush(h, (dist, node))                        # Dijkstra: tuples ★
heapq.heappush(h, (key, counter, obj))                 # tiebreak unorderable
heapq.heapify(a)  heapq.heappushpop(h, x)  heapq.heapreplace(h, x)
heapq.nlargest(k, a)  heapq.nsmallest(k, a, key=f)  heapq.merge(a, b)
from collections import deque
q = deque();  q = deque(a, maxlen=k)
q.append(x)  q.appendleft(x)  q.popleft()  q.pop()  q[0]  q[-1]  len(q)
q.rotate(k)  q.extend  q.extendleft          # BFS: popleft is O(1) ★
st = [];  st.append(x);  st.pop();  st[-1];  if st:   # stack = list
import queue; queue.Queue() / PriorityQueue()   # thread-safe only
```

18 Threads · Producer / Consumers — CONCURRENCY

```
import threading, queue
from concurrent.futures import ThreadPoolExecutor, as_completed
q = queue.Queue(maxsize=0)                  # thread-safe FIFO ★
results = []; lock = threading.Lock()
def worker():
    while True:
        j = q.get()                         # blocks
        if j is None: break                 # sentinel = stop ★
        with lock: results.append((j, j * j))
        q.task_done()
ts = [threading.Thread(target=worker, daemon=True) for _ in range(4)]
for t in ts: t.start()
for i in range(20): q.put(i)                # 1 producer
for _ in ts: q.put(None)                    # one sentinel per worker
for t in ts: t.join()
results.sort(key=lambda r: r[0])            # final sort ★
with ThreadPoolExecutor(max_workers=4) as ex:          # simpler ★
    results = list(ex.map(lambda j: (j, j * j), range(20)))   # keeps order
    futs = [ex.submit(f, j) for j in jobs]
    for fu in as_completed(futs): fu.result()
from multiprocessing import Pool            # CPU-bound: bypass the GIL
with Pool(4) as p: res = p.map(f, jobs)
import asyncio                              # I/O-bound
async def main(): r = await asyncio.gather(*(fetch(u) for u in urls))
asyncio.run(main())
```

Note Threads share memory but the GIL runs one at a time → threads help I/O, not CPU. `queue.Queue` is the safe hand-off; a bare list + `Lock` works for results.

19 Algorithm Templates — ALGORITHMS

```
# BFS on a grid
from collections import deque
dist = [[-1] * m for _ in range(n)]; dist[sr][sc] = 0
q = deque([(sr, sc)])
while q:
    r, c = q.popleft()
    for dr, dc in ((0, 1), (1, 0), (0, -1), (-1, 0)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < n and 0 <= nc < m and grid[nr][nc] != "#" \
                and dist[nr][nc] == -1:
            dist[nr][nc] = dist[r][c] + 1; q.append((nr, nc))
# Dijkstra
dist = [INF] * n; dist[s] = 0; pq = [(0, s)]
while pq:
    d, u = heapq.heappop(pq)
    if d > dist[u]: continue
    for v, w in adj[u]:
        if d + w < dist[v]: dist[v] = d + w; heapq.heappush(pq, (d + w, v))
# DFS recursive (set recursion limit!) / iterative
def dfs(u): vis[u] = True; [dfs(v) for v in adj[u] if not vis[v]]
st = [s]; vis = {s}
while st:
    u = st.pop()
    for v in adj[u]:
        if v not in vis: vis.add(v); st.append(v)
# Union-Find
par = list(range(n))
def find(x):
    while par[x] != x: par[x] = par[par[x]]; x = par[x]
    return x
def union(a, b): par[find(a)] = find(b)
# binary search on answer: smallest x with ok(x)
lo, hi = 0, 10**18
while lo < hi:
    mid = (lo + hi) // 2
    if ok(mid): hi = mid
    else: lo = mid + 1
# DP · prefix sum · sliding window
dp = [[0] * (m + 1) for _ in range(n + 1)]
pre = [0] + list(accumulate(a))            # sum(a[l:r]) = pre[r] - pre[l] ★
l = s = 0
for r, x in enumerate(a):
    s += x
    while s > k: s -= a[l]; l += 1
@lru_cache(None)
def go(i, j): ...                          # memo DP: tuple state ★
```

20 Misc · Easy to Forget — ALGORITHMS

`x & (x - 1)` `bin(x).count("1")` `x.bit_length()`	`x.bit_count()` /*3.10*/
`1 << k` `x >> 1` `~x` `x ^ y`	shift / not / xor
`math.gcd(a, b)` `math.lcm` `math.isqrt(n)`	`math.comb(n, k)` `math.factorial`
`pow(a, -1, m)`	modular inverse (3.8+)
`float("inf")` `-float("inf")` `sys.maxsize`	sentinels
`sorted(a) == sorted(b)` `Counter(a) == Counter(b)`	same elements?
`a == a[::-1]`	palindrome
`id(a) == id(b)` `a is b`	same object?
`python3 -c "..."` `python3 -m pdb x.py` `breakpoint()`	quick / debug

Routine ① `input = sys.stdin.readline` ② `setrecursionlimit` if recursing ③ `[[0]*m for _ in range(n)]` never `*n` ④ sort key tuple `(-score, name)` ⑤ `defaultdict` / `Counter` for counting ⑥ collect output, print once.